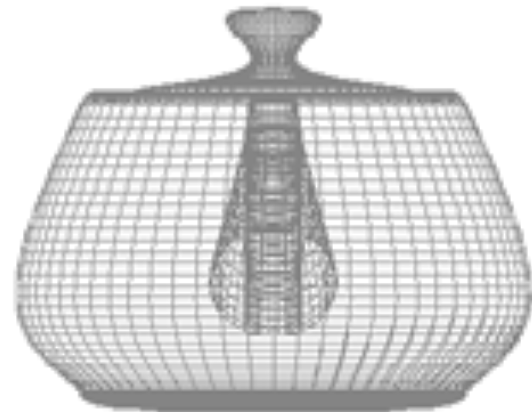
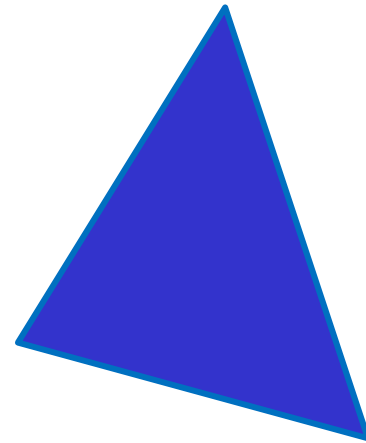
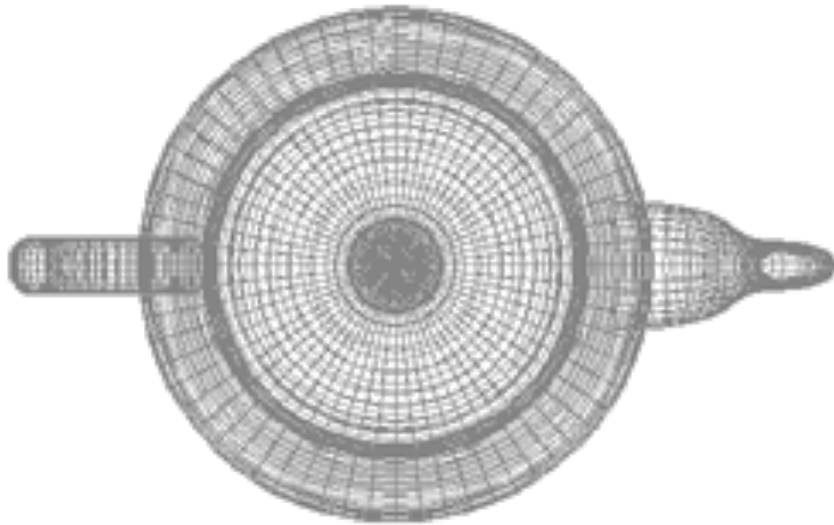


Laboratorio 4, parte 1

OpenGL Hello Triangle



Le librerie grafiche sono un delirio!

- Quale libreria?
 - Metal, Vulkan, OpenGL (innumerevoli tipi e versioni), Direct3D (11 o 12)...
 - Non è banale capire quale usare e perché.
 - Nessuna di queste è facile o intuitiva da usare.
- Scegliamo OpenGL 4.1!
 - Nonostante lo sviluppo di OpenGL sia oramai fermo, continua a far girare per applicazioni grafiche cross-platform (ultima versione: 4.6 del 2017, ma la 4.1, del 2010, è l'ultima supportata su MacOS).
 - Complicato fare il set-up e farraginoso da usare. Le alternative restano comunque peggiori!
- E per il futuro?
 - Andrebbe considerato WebGPU, un abstraction layer costruito sopra Vulkan, Metal e Direct3D12, pensato per i browser, ma con librerie native Rust e C++ per il desktop

Le librerie grafiche sono un delirio!

- Come mai è tutto così complicato?
 - Fare grafica 3D richiede di fare tanti, tanti calcoli. Per farli, bisogna usare hardware specializzato, molto complesso, con tante unità di computazione che lavorano in parallelo.
 - Sperare che il proprio software di grafica accelerata giri “correttamente” sul computer del primo che passa vuol dire dover prevedere un’enorme varietà:
 - Di schede video.
 - Ognuna con diverse versioni e tipologie di driver.
 - Sotto diversi sistemi operativi.
 - In OpenGL, questo vuol dire che il proprio programma deve chiedere al sistema cosa sia disponibile e caricare dinamicamente, a run time, i pezzi che gli servono. Per fortuna ci sono delle librerie che se ne occupano per noi. Noi useremo GLAD insieme a SFML.

Open GL: Macchina a stati

- OpenGL si basa su un proprio stato interno. Qualche esempio:
 - clear color attivo al momento.
 - depth test (abilitato o no, quale funzione usare).
 - blending (abilitato o no, quale funzione usare).
 - Quale buffer o spazio di memoria è al momento attivo per trasferire o modificare informazioni (aspetto importantissimo, in quanto ha a che fare con la comunicazione fra memoria di sistema e memoria video).
- Molti comandi non hanno effetto immediato, ma preparano lo stato influenzando quelli che poi saranno comandi successivi!
 - Lo stato interno contiene a tutti gli effetti dei parametri impliciti delle funzioni di libreria, questo rende il codice complesso da creare e seguire.
 - Esempio di query dello stato:

```
double v[4];  
glGetDoublev (GL_COLOR_CLEAR_VALUE, v);
```

Idea generale

- Geometria minimale: un triangolo, di un solo colore.
- **Vertex shader** minimale: passa il vertice tale e quale
- **Fragment shader** minimale: tutti i frammenti di quel colore.

1. Inizializzare OpenGL
2. Definire **cosa** disegnare
3. Definire **come** disegnare
4. Main loop con eventi base
5. . . . Disegnare



C++
OpenGL+GLSL
GLAD
SFML

Primo passo: inizializzare

- Potenzialmente, un incubo. Bisogna:
 - Includere OpenGL.
 - Farsi dare una finestra dal sistema, gestirne gli eventi (SFML).
 - Caricare dinamicamente i pezzi di OpenGL necessari (GLAD).
 - Inclusioni, inizializzazioni e caricamenti devono essere fatti in ordine corretto, altrimenti le cose smettono di funzionare!
 - Bisogna controllare cosa è stato effettivamente trovato e caricato sul sistema su cui sta girando il software, non c'è alcuna garanzia che le richieste siano esaudite.
 - In software più sofisticati, in questa fase si decide dinamicamente se abilitare o no certe feature in base a cosa il sistema supporta.
- **Compile il template di partenza, controllate se gira e cosa trova!!!**

Cenni su GLAD

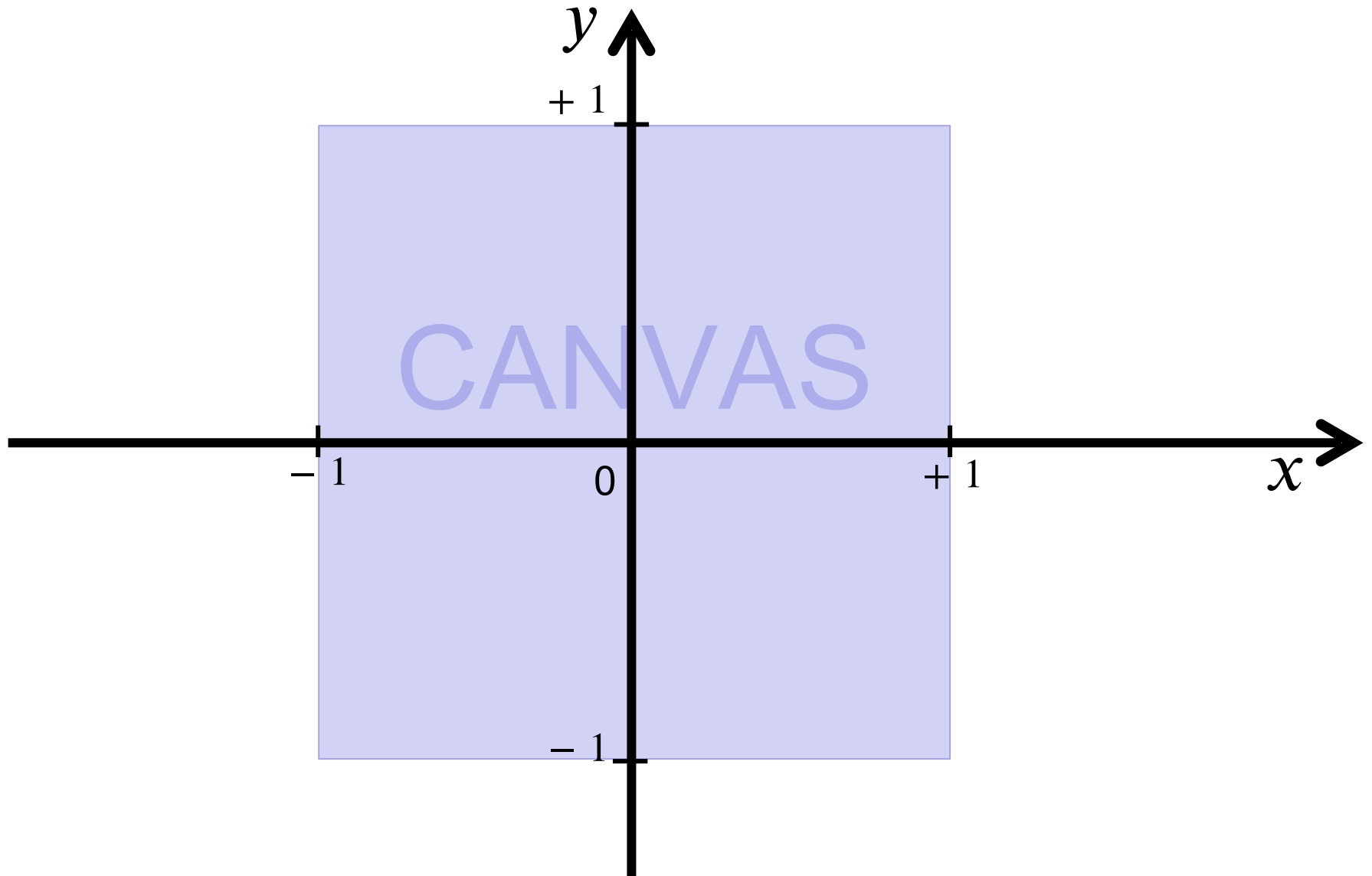
- Come funziona il sistema GLAD?
 - È un software open source, con un'interfaccia web, tramite la quale si chiede quale versione di OpenGL si vuole avere, e con quali estensioni. L'interfaccia genera una libreria (opzionalmente header only) da includere, che cerca di effettuare il caricamento dinamico di quanto richiesto.
 - Se in futuro aveste bisogno di una versione di OpenGL diversa, o estensioni specifiche, bisogna generare un nuovo header glad.
 - Al momento non c'è bisogno di fare nulla, forniamo noi un header glad già costruito, che carica una configurazione core 4.1, senza estensioni.
- In pratica, usate il template fornito: dovrebbe risolvere tutti questi problemi con un minimo comune denominatore accettabile.

Cosa disegniamo?

- Un solo triangolo in 2D.
- Direttamente in **clip coordinates** (nessuna trasformazione).
- Non indicizzato (soup), ovvero tre vertici consecutivi fanno un triangolo.
- In C++:

```
float points[] = {  
    0.0f,  0.5f,  0.0f,  // x,y,z of first point.  
    0.5f, -0.5f,  0.0f,  // x,y,z of second point.  
    -0.5f, -0.5f,  0.0f  // x,y,z of third point.  
};
```


Clip Space



Object Names (concetto generale)

- Abbiamo bisogno di oggetti nella RAM video:
 - Di solito, duplicati da oggetti nella RAM di sistema.
 - Per esempio, `vertex buffer`, `element buffer`, `programs`, `textures`, etc...
- Come li riferiamo, manipoliamo, e trasferiamo?
 - Letture, scritture devono passare attraverso il BUS di sistema!
- Si usano degli «**object names**»:
 - Un indice di una certa istanza di un dato oggetto, come un array buffer, etc.)
 - In pratica, un numero intero!
 - Il nome è in System RAM, quindi, semplicemente un intero in una variabile di programma) che punta ad un oggetto in Video RAM.
 - Il tipo di un oggetto è descritto da costanti dello stato di OpenGL, come `GL_CONSTANT_STUFF`.
 - I nomi degli oggetti sono generati da funzioni come «**create** <something>» o «**gen** <something>».
 - I nomi sono distrutti e liberati tramite funzioni come «**delete** <something>».

Binding (concetto generale)

- Situazione:
 - Molti comandi agiscono su un oggetto implicito nello stato di OpenGL.
 - Per esempio, su un **array buffer**.
 - Ci sono diverse istanze di quel tipo di oggetto.
 - Per esempio, diversi **array buffer** in memoria video.
- Come si sceglie su quale istanza agisce un comando?
 - In un dato momento, solo un'istanza è «**bound**» nello stato OpenGL!
 - Ad esempio, un **array buffer** è **bound**, un **program** è **bound**, etc.
 - Esiste un comando “**bind**” per attivare una specifica istanza nello stato.
 - Tutti i comandi successivi agiscono su quell'istanza, fino a quando non viene cambiata.

Caricamento dei dati in video RAM

- Generazione di **array buffer** e copia dei dati:

Vogliamo solo
un buffer.

Qui sarà restituito il **nome** del
buffer in Video RAM, da usare
per riferirlo in seguito.

GenBuffers: creazione!

```
glGenBuffers(1, &vbo);  
glBindBuffer(GL_ARRAY_BUFFER, vbo);  
glBufferData(GL_ARRAY_BUFFER,  
             sizeof(vertices),  
             vertices,  
             GL_STATIC_DRAW);
```

lunghezza e
puntatore ai dati in
system RAM da
copiare

BufferData: invio dei dati alla GPU

(suggerimento al driver video:
pensiamo di voler cambiare questi
dati solo raramente)

BindBuffer: tutti i comandi successivi
che usano un GL_ARRAY_BUFFER
implicito, useranno questo.

Organizzazione dei dati in video RAM

- Vertex Array Object:
 - Il VAO è un contenitore che descrive una parte dei dati che sono in memoria video e come interpretarli; per esempio, dove si trovano le coordinate e i colori di ogni singolo vertice.
 - Possono essercene più di uno: diversi VAO possono offrire viste differenti sull'insieme dei dati che sono stati trasferiti alla memoria video.
- In questo caso, aggiungiamo al VAO un solo attributo:
 - Le coordinate dei vertici, contenute nel VBO.

```
// Generate a vertex array object (VAO) named vao:
GLuint vao = 0;
glGenVertexArrays (1, &vao);

// Bind vao as the current VAO in the OpenGL state:
glBindVertexArray (vao);

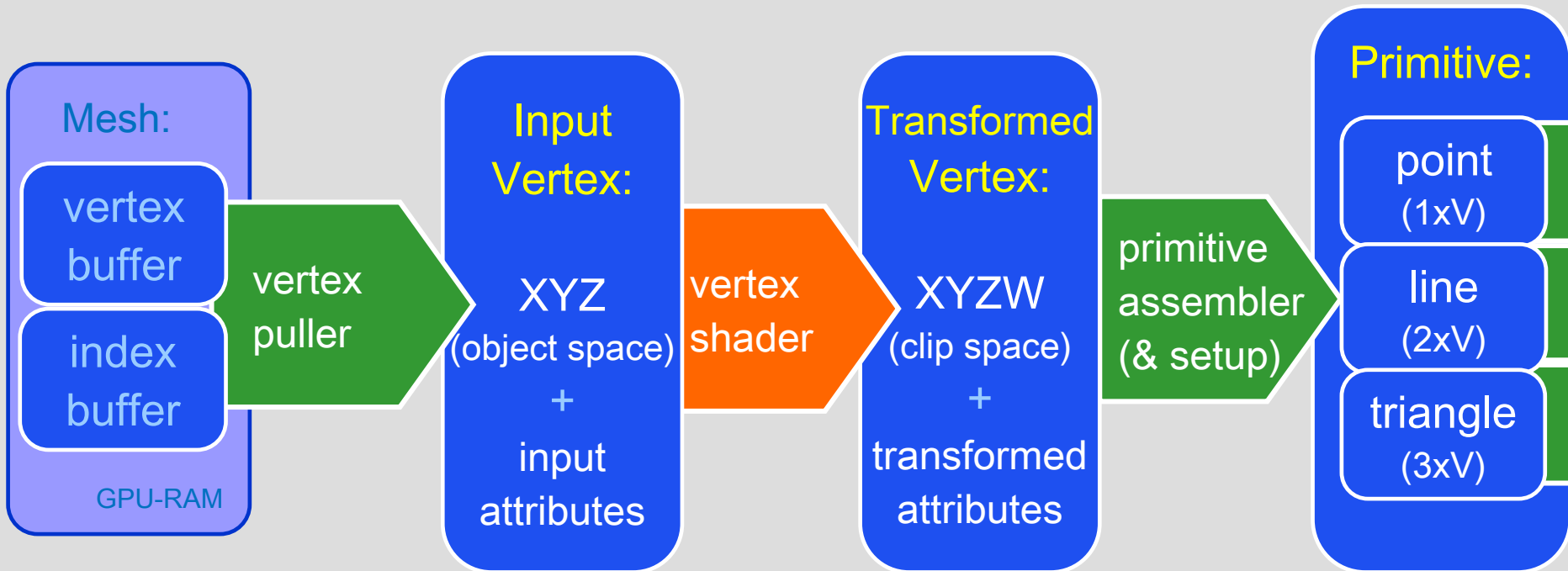
// Attach data inside current GL_ARRAY_BUFFER to current VAO,
// as attribute 0, where each one is made up of 3 floats, tightly packed:
glVertexAttribPointer (0, 3, GL_FLOAT, GL_FALSE, 0, NULL);

// Enable attribute 0 of current VAO for the vertex shader:
glEnableVertexAttribArray (0);
```

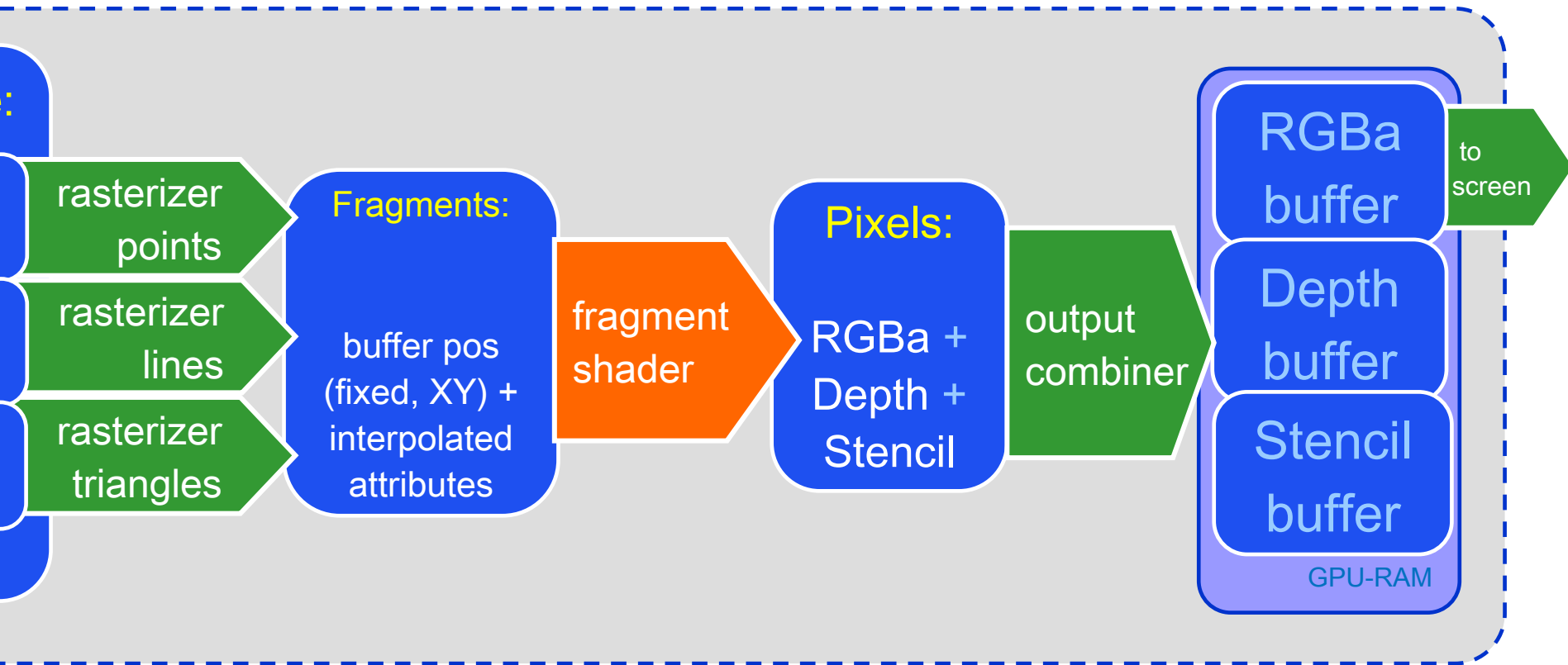
Come disegniamo?

- Definiamo gli shader!
 - **vertex shader** (viene eseguita un'istanza per ogni vertice).
 - **fragment shader** (viene eseguita un'istanza per ogni frammento).
- Linguaggio specifico: **GLSL**
 - «C-like»: niente oggetti.
 - Dinamicamente compilato in linguaggio macchina della GPU,
 - Ogni shader gira sulla GPU, al suo punto designato nella pipeline.
 - Tipi nativi vec2, vec3, vec4, mat4 e algebre corrispondenti.
- Nel nostro caso, shader minimali:
 - **vertex shader**: Passiamo le coordinate come le riceviamo (nessuna trasformazione, o meglio la trasformazione è l'identità).
 - **fragment shader**: Passiamo un colore costante.

Reminder: la pipeline grafica



Reminder: la pipeline grafica



Shader minimali

vertex shader:

VAO's attribute 0 is linked as **vp** in this shader's vertex

```
layout(location = 0) in vec2 vp;  
void main (void) {  
    gl_Position = vec4 (vp, 1.0);  
}
```

X Y Z

W (affine)

GLSL

fragment shader:

```
out vec4 FragColor;  
void main (void) {  
    FragColor = vec4 (0.0, 0.0, 1.0, 1.0);  
}
```

GLSL

Compilare gli shader

Gli shader vengono caricati nel programma come stringhe, compilati, passati alla GPU, ne viene fatto il linking e sono pronti per essere usati.

```
const char* vertex_shader = ". . ."; // Vertex shader code

const char* fragment_shader = ". . ."; // Fragment shader code

// Manage vertex shader
GLuint vert_s = glCreateShader (GL_VERTEX_SHADER); // Create
glShaderSource (vert_s, 1, &vertex_shader, NULL); // Attach source
glCompileShader (vert_s); // Compile

// Manage fragment shader
GLuint frag_s = glCreateShader (GL_FRAGMENT_SHADER); // Create
glShaderSource (frag_s, 1, &fragment_shader, NULL); // Attach source
glCompileShader (frag_s); // Compile

// Create a GPU program, attach to it both shaders, then link it.
GLuint shader_program = glCreateProgram();
glAttachShader (shader_program, frag_s);
glAttachShader (shader_program, vert_s);
glLinkProgram (shader_program);

// The shaders are now active inside the GPU pipeline!
```

... Finalmente si disegna!

```
while (open_window) {  
    // Clear the color buffer (default black)  
    glClear(GL_COLOR_BUFFER_BIT);  
  
    // The current vertex array is the one used for rendering  
    glBindVertexArray(vao);  
  
    // Render!  
    glDrawArrays(GL_TRIANGLES, 0, 3);  
}
```

- **glDrawArrays**

- Manda alla pipeline i dati del current VAO, assemblandoli in primitive del tipo GL_TRIANGLES, partendo dal primo vertice, e mandando 3 vertici in totale.
- Con glDrawArrays si potrebbe disegnare anche un sottoinsieme dei vertici (partendo oltre lo zero e/o dando un numero di vertici minore del totale).